WP1 — Mathematical Foundations

The BJKST Estimator for the Distinct-Elements Problem in the Streaming Model

Raunak Kumar Rajak

September 21, 2026

Abstract

This document gives a self-contained, mathematically rigorous treatment of the BJKST algorithm of Bar-Yossef, Jayram, Kumar, Sivakumar and Trevisan for estimating the number of distinct elements F_0 in a data stream. We begin with the streaming model and the frequency moments F_k, develop the probabilistic tools (Chebyshev, Hoeffding) and the hashing tools (pairwise independence) that are needed, and then derive the estimator from the idea of *sampling by hash value*. We prove that a single copy of the algorithm returns a $(1 \pm \varepsilon)$-approximation with probability at least $2/3$, analyse its space complexity in two forms (storing raw identifiers, and storing short fingerprints), and boost the success probability to $1 - \delta$ by the median trick. Section 1 lists the papers, books and lecture notes on which each part of the theory rests.

Contents

1 Sources and Supporting Literature **2**

2 The Streaming Model and the Distinct-Elements Problem **2**
2.1 The streaming model . . . 2
2.2 Frequency moments . . . 3
2.3 Why randomization and approximation are needed . . . 3

3 Probabilistic Tools **3**

4 Hash Functions and Pairwise Independence **4**
4.1 Definitions . . . 4
4.2 An explicit pairwise independent family . . . 4
4.3 Trailing zeros . . . 4

5 Randomized Sampling by Hash Value and the Estimator **4**
5.1 Sampling the *distinct* elements . . . 4
5.2 The estimator and why it is natural . . . 5

6 The BJKST Algorithm **5**
6.1 Correctness invariant . . . 5

7 Accuracy Analysis **6**

8 Space Complexity **7**
8.1 Identifier-storing form . . . 7
8.2 Fingerprint form . . . 8
8.3 Update time . . . 8

9 Boosting the Success Probability **8**

10 Discussion: Context and Optimality **9**

1 Sources and Supporting Literature

The theory in this document is not original; it is a careful re-derivation of standard results. The table below records where each part comes from. Full bibliographic details are in the References. Chapter-level pointers are given for books; please verify section and page numbers against the specific edition you use.

Topic in this document	Supporting source(s)
BJKST algorithm and its guarantees	Bar-Yossef, Jayram, Kumar, Sivakumar, Trevisan, *Counting Distinct Elements in a Data Stream*, RANDOM 2002 [1] (primary source). The buffer-with-threshold formulation analysed here follows the style of lecture-note treatments, e.g. Chakrabarti, *Data Stream Algorithms* [12].
Streaming model, frequency moments F_k, lower bounds	Alon, Matias, Szegedy, *The Space Complexity of Approximating the Frequency Moments* [2]; Muthukrishnan, *Data Streams: Algorithms and Applications* [11]; Leskovec, Rajaraman, Ullman, *Mining of Massive Datasets*, Ch. 4 [9].
Probabilistic counting idea (trailing zeros of a hash)	Flajolet, Martin, *Probabilistic Counting Algorithms for Data Base Applications* [3]; [9], Ch. 4.
Markov, Chebyshev, Chernoff–Hoeffding bounds	Mitzenmacher, Upfal, *Probability and Computing*, 2nd ed., Ch. 3–4 [6]; Motwani, Raghavan, *Randomized Algorithms* [7].
Hash functions, universality, pairwise independence	Carter, Wegman [4]; Wegman, Carter [5]; Cormen, Leiserson, Rivest, Stein, *Introduction to Algorithms*, Ch. 11 [8]; Vadhan, *Pseudorandomness* [13]; [6]; [7].
Median trick for confidence amplification	[2]; [6]; [7].
Optimality and lower bounds for F_0	Indyk, Woodruff [14]; Woodruff [15]; Kane, Nelson, Woodruff [16].
Book-length treatments of stream summaries	Cormode, Yi, *Small Summaries for Big Data* [10].

2 The Streaming Model and the Distinct-Elements Problem

2.1 The streaming model

Let $[n] = \{0, 1, \ldots, n-1\}$ be a universe of items. A *data stream* is a sequence $\sigma = (a_1, a_2, \ldots, a_m)$ with each $a_t \in [n]$, presented one item at a time. A streaming algorithm may

- make a single pass over σ (in the basic model),
- use memory that is *sublinear* in m and n, ideally $\mathrm{polylog}(m, n)$ or $\mathrm{poly}(1/\varepsilon) \cdot \mathrm{polylog}$,
- spend little time per item.

The complexity measure of interest is the number of *bits* of working memory. See [2, 11, 9, 10]. Throughout we assume, without loss of generality, that $n = 2^b$ is a power of two (pad the universe otherwise).

2.2 Frequency moments

For $i \in [n]$ let $f_i = |\{t : a_t = i\}|$ be the frequency of i. The *k-th frequency moment* is

$$F_k \;=\; \sum_{i \in [n]} f_i^{\,k}, \qquad k \geq 0, \quad \text{with the convention } 0^0 = 0.$$

Thus $F_1 = m$ (stream length), F_2 is the "repeat rate" (surprise number), and

$$F_0 \;=\; \big|\{i : f_i > 0\}\big| \;=:\; d$$

is the number of *distinct* elements. Frequency moments were introduced as a streaming problem in [2].

Definition 2.1 ((ε, δ)-approximation)**.** A randomized algorithm outputs an (ε, δ)-approximation of d if its output $\hat{d}$ satisfies $\mathbb{P}\big[\,|\hat{d} - d| \leq \varepsilon d\,\big] \geq 1 - \delta$, the probability being over the algorithm's internal randomness (not over the input, which is adversarial but fixed in advance).

2.3 Why randomization and approximation are needed

Computing d *exactly* needs $\Omega(\min\{n, m\})$ bits of memory, because the algorithm must in effect remember which items have appeared. Alon, Matias and Szegedy [2] show that the same $\Omega(n)$ barrier applies to *deterministic* algorithms that only approximate F_0 (to within a fixed constant factor). Hence both randomization *and* approximation are essential to obtain polylogarithmic-space algorithms.

3 Probabilistic Tools

All results below are standard; see [6, 7].

Lemma 3.1 (Markov)**.** *If $X \geq 0$ and $a > 0$, then $\mathbb{P}[X \geq a] \leq \mathbb{E}[X]/a$.*

Lemma 3.2 (Chebyshev)**.** *For any random variable X with finite variance and any $t > 0$, $\mathbb{P}\big[|X - \mathbb{E}X| \geq t\big] \leq \mathrm{Var}[X]/t^2$.*

Lemma 3.3 (Variance of a sum of pairwise independent variables)**.** *If $X_1, \ldots, X_d$ are pairwise independent, then $\mathrm{Var}\big[\sum_i X_i\big] = \sum_i \mathrm{Var}[X_i]$.*

Proof. Expanding the variance,

$$\mathrm{Var}\Big[\sum_i X_i\Big] = \sum_i \mathrm{Var}[X_i] + \sum_{i \neq j} \mathrm{Cov}(X_i, X_j),$$

and pairwise independence gives $\mathrm{Cov}(X_i, X_j) = 0$ for $i \neq j$. □

Lemma 3.4 (Hoeffding)**.** *Let $W_1, \ldots, W_k$ be independent $\{0,1\}$-valued random variables and $W = \sum_j W_j$. Then for every $t > 0$, $\mathbb{P}[W - \mathbb{E}W \geq t] \leq \exp(-2t^2/k)$.*

Only the second moment (Chebyshev) is needed to analyse a *single* copy of BJKST; Hoeffding is used only for amplifying the success probability (Section 9).

4 Hash Functions and Pairwise Independence

4.1 Definitions

Let H be a family of functions $h : U \to R$, and let h be drawn uniformly from H.

Definition 4.1 (Pairwise independent (strongly 2-universal) family)**.** H is *pairwise independent* if for all distinct $x, y \in U$ and all $r, s \in R$,

$$\mathbb{P}_{h \in H}\big[h(x) = r \ \wedge \ h(y) = s\big] = \frac{1}{|R|^2}.$$

Equivalently: for each x, $h(x)$ is uniform on R, and for distinct x, y the values $h(x), h(y)$ are independent.

Definition 4.2 (2-universal family)**.** H is 2-*universal* if $\mathbb{P}_{h \in H}[h(x) = h(y)] \le 1/|R|$ for all distinct $x, y \in U$.

Pairwise independence implies 2-universality. These notions are due to Carter and Wegman [4, 5]; see also [8, Ch. 11] and [13].

4.2 An explicit pairwise independent family

Let $\mathbb{F} = \mathrm{GF}(2^b)$, whose elements we identify with b-bit strings (hence with $[n]$ when $n = 2^b$). Define

$$H = \big\{\, h_{a,c} : \mathbb{F} \to \mathbb{F},\ h_{a,c}(x) = a \cdot x + c \ \ : \ \ a, c \in \mathbb{F} \,\big\}.$$

Lemma 4.3. *H is a pairwise independent family of functions $[n] \to [n]$. Choosing $h \in H$ requires $2b = 2\log_2 n$ random bits, which is also the description length of h.*

Proof. Fix distinct $x, y \in \mathbb{F}$ and targets $r, s \in \mathbb{F}$. The system $ax + c = r$, $ay + c = s$ has the unique solution $a = (r - s)/(x - y)$, $c = r - ax$, because $x - y \neq 0$ is invertible in a field. So exactly one pair (a, c) out of $|\mathbb{F}|^2 = n^2$ realises $(h(x), h(y)) = (r, s)$, so the probability is $1/n^2$. □

Remark 4.4. Over a prime field $\mathbb{Z}_p$ the same map $x \mapsto ax + c \bmod p$ is pairwise independent into $\mathbb{Z}_p$, but then the range is not a power of two, and the "trailing zeros" events below have probability only approximately 2^{-z}. Using $\mathrm{GF}(2^b)$ makes the probabilities exact.

4.3 Trailing zeros

For $x \in \{0, \ldots, 2^b - 1\}$ let $\mathrm{zeros}(x)$ be the number of trailing zeros of its b-bit binary expansion, with $\mathrm{zeros}(0) = b$.

Lemma 4.5. *If X is uniform on $\{0, \ldots, 2^b - 1\}$, then for every integer $0 \le z \le b$,*

$$\mathbb{P}[\mathrm{zeros}(X) \ge z] \ = \ 2^{-z}.$$

Proof. $\mathrm{zeros}(X) \ge z$ iff the lowest z bits of X are all zero, iff $2^z \mid X$. Exactly 2^{b-z} of the 2^b values are multiples of 2^z. □

5 Randomized Sampling by Hash Value and the Estimator

5.1 Sampling the *distinct* elements

Let $D \subseteq [n]$ be the set of distinct items in the stream, $|D| = d$. Fix a pairwise independent $h : [n] \to [n]$ and, for each level $z \in \{0, \ldots, b\}$, define the *level-z sample*

$$S_z \ = \ \{\, a \in D : \ \mathrm{zeros}(h(a)) \ge z \,\}, \qquad Y_z = |S_z|.$$

Two features make this a good sampling scheme.

Probability of retention. By Lemma 4.3 and Lemma 4.5, every item a is retained in S_z with probability exactly $p_z = 2^{-z}$, and, for distinct items $a \neq a'$, the events $\{a \in S_z\}$, $\{a' \in S_z\}$ are independent. Retention is determined by the *value* a (through $h(a)$), not by the position of an occurrence in the stream. Consequently an item that appears f_a times is either retained at all its occurrences or at none, so duplicates neither inflate nor bias the sample. This is what makes the sampled quantity a count of *distinct* elements.

Nested samples. $\text{zeros}(h(a)) \geq z+1 \Rightarrow \text{zeros}(h(a)) \geq z$, so $D = S_0 \supseteq S_1 \supseteq S_2 \supseteq \cdots$. The level-$(z+1)$ sample is obtained from the level-z sample by discarding those items whose stored zeros-value equals exactly z (each survives independently, in the pairwise sense, with conditional probability $1/2$). This permits the algorithm to *tighten* its sample on the fly without re-reading the stream.

5.2 The estimator and why it is natural

Define

$$\hat{d}_z \;=\; 2^z\, Y_z.$$

Proposition 5.1 (Unbiasedness and variance)**.** *For every $0 \leq z \leq b$, with $\mu_z := \mathbb{E}[Y_z] = d/2^z$,*

$$\mathbb{E}[\hat{d}_z] = d, \qquad \text{Var}[Y_z] = d\,2^{-z}(1-2^{-z}) \leq \mu_z, \qquad \text{Var}[\hat{d}_z] \leq 2^z d.$$

Proof. Write $Y_z = \sum_{a \in D} X_a$ with $X_a = \mathbf{1}[\text{zeros}(h(a)) \geq z]$, a Bernoulli (2^{-z}) variable. Linearity gives $\mathbb{E}[Y_z] = d2^{-z}$. The X_a are pairwise independent, so Lemma 3.3 yields $\text{Var}[Y_z] = d\,2^{-z}(1-2^{-z})$. Finally $\text{Var}[\hat{d}_z] = 4^z \,\text{Var}[Y_z] \leq 4^z \cdot d2^{-z} = 2^z d$. □

Corollary 5.2 (Chebyshev bound at a fixed level)**.** *For every fixed level z and $\varepsilon \in (0,1)$,*

$$\mathbb{P}\big[\,|Y_z - \mu_z| > \varepsilon\mu_z\,\big] \;\leq\; \frac{\text{Var}[Y_z]}{\varepsilon^2\mu_z^2} \;\leq\; \frac{1}{\varepsilon^2\mu_z}.$$

The design principle. Corollary 5.2 shows that the estimator $\hat{d}_z$ is ε-accurate with good probability as soon as $\mu_z = \mathbb{E}[Y_z] \gtrsim 1/\varepsilon^2$. But memory is proportional to Y_z, so we want the *largest* z (smallest sample) that still has $\mu_z \gtrsim 1/\varepsilon^2$. Since d is unknown, the right z is unknown. BJKST resolves this by starting at $z = 0$ and increasing z each time the buffer grows past a threshold $\tau = \Theta(1/\varepsilon^2)$. Because the samples are nested, this "self-tuning" costs nothing.

6 The BJKST Algorithm

The original paper [1] presents three algorithms. The version below is the buffer-with-threshold sampling formulation (see also [12]); the identifier-storing form is presented first, and a compressed form is analysed in Section 8.

6.1 Correctness invariant

Lemma 6.1 (Invariant)**.** *Let D_t be the set of distinct items among $a_1, \ldots, a_t$. At all times during the processing of a_t, immediately after the insertion step and immediately after each pruning step, the buffer satisfies*

$$B = \{(a, \text{zeros}(h(a))) :\; a \in D_t,\; \text{zeros}(h(a)) \geq z\}.$$

Algorithm 1 BJKST (identifier-storing form)

Require: Stream $a_1, \ldots, a_m \in [n]$, $n = 2^b$; accuracy $\varepsilon \in (0, 1)$

$\tau \leftarrow \lceil c/\varepsilon^2 \rceil$ ▷ $c = 60$ suffices; see Theorem 7.1
Choose h uniformly from the pairwise independent family of Lemma 4.3
$B \leftarrow \emptyset, \quad z \leftarrow 0$
for each item a of the stream **do**
  if $\text{zeros}(h(a)) \geq z$ **then**
    $B \leftarrow B \cup \{(a, \text{zeros}(h(a)))\}$ ▷ B is a set, so a repeated a is a no-op
  end if
  while $|B| \geq \tau$ **do**
    $z \leftarrow z + 1$
    $B \leftarrow \{(x, \beta) \in B : \beta \geq z\}$
  end while
end for
return $\hat{d} = |B| \cdot 2^z$

Proof. Induction over the steps. Initially $B = \emptyset$, $z = 0$, $D_0 = \emptyset$. Insertion of a_t adds its pair exactly when $\text{zeros}(h(a_t)) \geq z$; since the pair is a function of a_t alone, re-inserting a repeated item changes nothing. Pruning from level z to $z+1$ removes exactly the pairs with $\beta = z$, which is precisely the difference between the level-z and level-$(z+1)$ sets. A pair removed at some level z' has $\text{zeros} < z' \leq z$ for all later z, so it never qualifies for reinsertion. □

Corollary 6.2 (Final state)**.** *Let Z denote the final value of z and let Y_z be as in Section 5, with $Y_z = 0$ for $z > b$. Then, deterministically,*

$$|B| = Y_Z < \tau, \qquad \text{and} \qquad Y_z \geq \tau \quad \text{for all } z < Z,$$

that is, $Z = \min\{z \geq 0 : \ Y_z < \tau\}$, and the output is $\hat{d} = 2^Z Y_Z = \hat{d}_Z$.

Proof. After the last while-loop, $|B| < \tau$, and by Lemma 6.1 at the end of the stream $|B| = Y_Z$. For $z < Z$, the algorithm left level z at some time t because $|B| \geq \tau$; by the invariant, B then equaled $\{a \in D_t : \text{zeros}(h(a)) \geq z\} \subseteq S_z$, so $Y_z \geq \tau$. □

Corollary 6.2 is the key structural fact: the (random) final level Z is the first level at which the *full-stream* sample S_z becomes smaller than τ, and the output is exactly the estimator $\hat{d}_Z$ of Section 5 evaluated at that level. The subtlety is that Z is random, so Corollary 5.2 cannot be applied at Z directly. We handle this by a union bound over the levels that can matter.

7 Accuracy Analysis

Theorem 7.1 (Accuracy of one copy)**.** *Let $0 < \varepsilon < 1$, $c = 60$ and $\tau = \lceil c/\varepsilon^2 \rceil$. Then Algorithm 1 outputs $\hat{d}$ with*

$$\mathbb{P}\big[\, |\hat{d} - d| \leq \varepsilon d \,\big] \ \geq \ \frac{5}{6}.$$

Proof. Write $\mu_z = d/2^z$. If $d < \tau/2$ then $Y_0 = d < \tau$, the algorithm never increments z, $Z = 0$, and the output is exactly d. Assume $d \geq \tau/2$ and define the *target level*

$$z_* \ = \ \min\{z \geq 0 : \ \mu_z < \tau/2\}.$$

Since $\mu_b = d/n \leq 1 < \tau/2$, we have $z_* \leq b$. By minimality and halving, if $z_* \geq 1$ then $\mu_{z_*-1} \geq \tau/2$, hence

$$\frac{\tau}{4} \ \leq \ \mu_{z_*} \ < \ \frac{\tau}{2}. \tag{1}$$

(If $z_* = 0$ then $Y_0 = \mu_0 = d$ is deterministic and the same conclusion below holds trivially with failure probability 0, because $Y_0 = d < \tau/2 < \tau$ forces $Z = 0$.) Assume $z_* \geq 1$ from now on.

By Corollary 6.2, the output is wrong, i.e. $|2^Z Y_Z - d| > \varepsilon d$, only if $|Y_Z - \mu_Z| > \varepsilon \mu_Z$ (multiply by 2^{-Z}). Hence the failure event is contained in $\mathcal{E}_1 \cup \mathcal{E}_2$ where

$$\mathcal{E}_1 = \{Z \geq z_* + 1\}, \qquad \mathcal{E}_2 = \{\exists z \leq z_* : \ |Y_z - \mu_z| > \varepsilon \mu_z\}.$$

(Indeed if $Z \leq z_*$ and the output is wrong, take $z = Z$.)

Bounding $\mathcal{E}_1$ (level overshoots). By Corollary 6.2, $Z \geq z_* + 1$ implies $Y_{z_*} \geq \tau$. Since $\mu_{z_*} < \tau/2$, this means $Y_{z_*} - \mu_{z_*} > \tau - \mu_{z_*} > \tau/2$. Chebyshev with $\mathrm{Var}[Y_{z_*}] \leq \mu_{z_*}$ (Proposition 5.1) gives

$$\mathbb{P}[\mathcal{E}_1] \ \leq \ \frac{\mu_{z_*}}{(\tau - \mu_{z_*})^2} \ \leq \ \frac{\tau/2}{(\tau/2)^2} \ = \ \frac{2}{\tau} \ \leq \ \frac{2}{c},$$

where the middle inequality uses that $\mu \mapsto \mu/(\tau - \mu)^2$ is increasing on $(0, \tau)$, and the last uses $\tau \geq c/\varepsilon^2 \geq c$.

Bounding $\mathcal{E}_2$ (bad concentration at low levels). By a union bound and Corollary 5.2,

$$\mathbb{P}[\mathcal{E}_2] \leq \sum_{z=0}^{z_*} \frac{1}{\varepsilon^2 \mu_z}.$$

Because $\mu_z = \mu_{z_*} 2^{z_* - z}$, the sum is geometric:

$$\sum_{z=0}^{z_*} \frac{1}{\varepsilon^2 \mu_z} = \frac{1}{\varepsilon^2 \mu_{z_*}} \sum_{j=0}^{z_*} 2^{-j} < \frac{2}{\varepsilon^2 \mu_{z_*}} \leq \frac{8}{\varepsilon^2 \tau} \leq \frac{8}{c},$$

using (1) and $\tau \geq c/\varepsilon^2$.

Conclusion. $\mathbb{P}[\text{fail}] \leq \mathbb{P}[\mathcal{E}_1] + \mathbb{P}[\mathcal{E}_2] \leq 10/c = 1/6$ for $c = 60$. □

Remark 7.2 (Why the argument is shaped this way). The output is $\hat{d}_Z$ at a *random* level Z, so one cannot simply apply Chebyshev at Z. The proof circumvents this by (i) showing Z is, with high probability, not larger than the deterministic target z_*, where $\mu_{z_*} \approx \tau/2\Theta(1) = \Theta(1/\varepsilon^2)$, and (ii) taking a union bound of the level-wise Chebyshev failures over all levels $\leq z_*$. Because μ_z doubles each level moving down, the failure probabilities form a geometric series that is dominated by its last term, so the union bound costs only a constant factor. Only *pairwise* independence of h was used.

8 Space Complexity

8.1 Identifier-storing form

By Corollary 6.2 and the loop condition, $|B| < \tau$ at the end of each item's processing, and $|B| \leq \tau$ at all times. Each entry stores an identifier ($b = \log_2 n$ bits) and a zeros-value (at most $\lceil \log_2(b+1) \rceil$ bits); the counter z needs $\mathcal{O}(\log b)$ bits and the hash h needs $2 \log_2 n$ bits (Lemma 4.3). Total:

$$\mathcal{O}\Big(\frac{1}{\varepsilon^2} \log n\Big) \text{ bits.}$$

8.2 Fingerprint form

The dominant cost is the identifiers. They can be replaced by short *fingerprints*: choose a second, independent 2-universal function $g : [n] \to [M]$ and store the pair $(g(a), \text{zeros}(h(a)))$ instead of $(a, \text{zeros}(h(a)))$. Duplicate detection is then done on fingerprints. The danger is a false collision, which would make a new item look like a duplicate and cause undercounting.

Lemma 8.1 (Few items ever enter the buffer)**.** *Let I be the set of distinct items ever inserted into B by the identifier-storing algorithm. Then, deterministically, $|I| \le \tau(b+1)$.*

Proof. Insertions occur only at levels $0, \dots, b$ (no item has zeros $> b$). At any level the buffer starts with fewer than τ elements, each insertion increases $|B|$ by one, and the level is left as soon as $|B| = \tau$; so at most τ insertions occur per level. A pruned item is never reinserted (Lemma 6.1), so no item is counted twice. □

Theorem 8.2 (Space of the fingerprint form)**.** *Take $M = \lceil 10\,\tau^2 (b+1)^2 \rceil$. With g independent of h, the fingerprint form of the algorithm outputs a $(1 \pm \varepsilon)$-approximation with probability at least $2/3$ and uses*

$$\mathcal{O}\Big(\log n + \frac{1}{\varepsilon^2}\big(\log \tfrac{1}{\varepsilon} + \log\log n\big)\Big)$$

bits of space.

Proof. Condition on h. As long as g has no collision among the items of I, the fingerprint run is step-for-step identical to the identifier-storing run, and therefore has the same output. The set I is determined by h and the stream alone, and g is independent of h, so by 2-universality and Lemma 8.1,

$$\mathbb{P}[\exists\, x \neq y \in I :\ g(x) = g(y)] \le \binom{|I|}{2}\frac{1}{M} \le \frac{\tau^2(b+1)^2}{2M} \le \frac{1}{20}.$$

By Theorem 7.1, the total failure probability is at most $\frac{1}{6} + \frac{1}{20} = \frac{13}{60} < \frac{1}{3}$.

Space. Each stored entry costs $\lceil \log_2 M \rceil + \lceil \log_2 (b+1) \rceil$ bits. Now $\log M = \mathcal{O}(\log \tau + \log b) = \mathcal{O}(\log \frac{1}{\varepsilon} + \log\log n)$, so the buffer takes $\mathcal{O}\big(\tau(\log \frac{1}{\varepsilon} + \log\log n)\big)$ bits. The two hash functions (h into $[n]$ and g into $[M]$, with $M \le \text{poly}(n)$) are described in $\mathcal{O}(\log n)$ bits, and z takes $\mathcal{O}(\log\log n)$ bits. □

8.3 Update time

Each item costs $\mathcal{O}(1)$ hash evaluations (field operations in $\text{GF}(2^b)$) and a dictionary lookup. Pruning rescans B but is triggered at most $b+1$ times, and each level's scan is charged to the at most τ insertions made at that level, giving $\mathcal{O}(1)$ expected (amortised) time per item with a hash table.

9 Boosting the Success Probability

Theorem 9.1 (Median trick)**.** *Run $k = \lceil 18 \ln(1/\delta) \rceil$ independent copies of the algorithm (independent choices of h, g) and output the median $\tilde{d}$ of their outputs. Then $\mathbb{P}\big[|\tilde{d} - d| \le \varepsilon d\big] \ge 1 - \delta$.*

Proof. Let $W_j = 1$ if copy j is wrong, i.e. its output lies outside $[(1-\varepsilon)d, (1+\varepsilon)d]$. The W_j are independent with $\mathbb{P}[W_j = 1] \le \frac{1}{3}$, so $\mathbb{E}[W] \le k/3$ for $W = \sum_j W_j$. If fewer than $k/2$ copies are wrong, then more than half of the outputs lie in the interval, and so does their median. Therefore the median fails only if $W \ge k/2$, and by Lemma 3.4 with $t = k/6$,

$$\mathbb{P}[W \ge k/2] \le \mathbb{P}[W - \mathbb{E}W \ge k/6] \le \exp\Big(-\frac{2(k/6)^2}{k}\Big) = e^{-k/18} \le \delta.$$

(The case $\mathbb{P}[W_j = 1] < 1/3$ is handled by stochastic domination.) □

Corollary 9.2 (Overall guarantee)**.** *There is a one-pass randomized streaming algorithm which, for any $\varepsilon, \delta \in (0,1)$, outputs an (ε,δ)-approximation of F_0 using*

$$\mathcal{O}\Big(\log \tfrac{1}{\delta} \cdot \Big(\log n + \tfrac{1}{\varepsilon^2}\big(\log \tfrac{1}{\varepsilon} + \log\log n\big)\Big)\Big)$$

bits of space.

10 Discussion: Context and Optimality

Relation to Flajolet–Martin. The trailing-zeros idea comes from Flajolet and Martin [3], who track the *maximum* $\mathrm{zeros}(h(a))$ observed; that maximum is around $\log_2 d$, giving a constant-factor estimate under idealized (fully random) hashing. Alon, Matias and Szegedy [2] showed that pairwise independent hashing suffices for a constant-factor approximation. BJKST retains a *sample* of items instead of just a maximum, which is what buys $(1\pm\varepsilon)$ accuracy while needing only pairwise independence.

Relation to k-minimum-value sketches. Another standard presentation of the same idea in [1] keeps the $t = \Theta(1/\varepsilon^2)$ smallest hash values and estimates d from the t-th smallest. Both variants are instances of "sample by hash value, calibrate by the retention probability".

Optimality. Any randomized $(\varepsilon, 1/3)$-approximation of F_0 needs $\Omega(1/\varepsilon^2)$ bits (Indyk–Woodruff [14], Woodruff [15]) and $\Omega(\log n)$ bits. Kane, Nelson and Woodruff [16] gave an algorithm using $\mathcal{O}(1/\varepsilon^2 + \log n)$ bits, matching these bounds; the BJKST bound in Theorem 8.2 is within a $\log(1/\varepsilon) + \log\log n$ factor of the $1/\varepsilon^2$ term.

References

[1] Z. Bar-Yossef, T. S. Jayram, R. Kumar, D. Sivakumar, L. Trevisan. *Counting Distinct Elements in a Data Stream.* In *Randomization and Approximation Techniques in Computer Science (RANDOM 2002)*, LNCS vol. 2483, pp. 1–10, Springer, 2002.

[2] N. Alon, Y. Matias, M. Szegedy. *The Space Complexity of Approximating the Frequency Moments.* Journal of Computer and System Sciences 58(1):137–147, 1999 (preliminary version in STOC 1996).

[3] P. Flajolet, G. N. Martin. *Probabilistic Counting Algorithms for Data Base Applications.* Journal of Computer and System Sciences 31(2):182–209, 1985.

[4] J. L. Carter, M. N. Wegman. *Universal Classes of Hash Functions.* Journal of Computer and System Sciences 18(2):143–154, 1979.

[5] M. N. Wegman, J. L. Carter. *New Hash Functions and Their Use in Authentication and Set Equality.* Journal of Computer and System Sciences 22(3):265–279, 1981.

[6] M. Mitzenmacher, E. Upfal. *Probability and Computing: Randomization and Probabilistic Techniques in Algorithms and Data Analysis*, 2nd edition. Cambridge University Press, 2017.

[7] R. Motwani, P. Raghavan. *Randomized Algorithms.* Cambridge University Press, 1995.

[8] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to Algorithms*, 4th edition. MIT Press, 2022.

[9] J. Leskovec, A. Rajaraman, J. D. Ullman. *Mining of Massive Datasets*, 3rd edition. Cambridge University Press, 2020. (Chapter 4, "Mining Data Streams".)

[10] G. Cormode, K. Yi. *Small Summaries for Big Data.* Cambridge University Press, 2020.

[11] S. Muthukrishnan. *Data Streams: Algorithms and Applications.* Foundations and Trends in Theoretical Computer Science 1(2):117–236, 2005.

[12] A. Chakrabarti. *Data Stream Algorithms* (lecture notes). Dartmouth College.

[13] S. P. Vadhan. *Pseudorandomness.* Foundations and Trends in Theoretical Computer Science 7(1–3):1–336, 2012.

[14] P. Indyk, D. P. Woodruff. *Tight Lower Bounds for the Distinct Elements Problem.* In Proc. 44th IEEE FOCS, pp. 283–288, 2003.

[15] D. P. Woodruff. *Optimal Space Lower Bounds for All Frequency Moments.* In Proc. 15th ACM-SIAM SODA, pp. 167–175, 2004.

[16] D. M. Kane, J. Nelson, D. P. Woodruff. *An Optimal Algorithm for the Distinct Elements Problem.* In Proc. 29th ACM PODS, pp. 41–52, 2010.